n8n engineer prep: the questions I got asked, answered with what is in this repo

Short answers first, then the detail. Every answer points at something you can run or open here.

"Do you run it in Docker? Locally? In the cloud?"

Three ways to run n8n, and I have used two of them:

1. **npm on a laptop** (what this repo does). `npm install n8n`, `n8n start`, it listens on port 5678 with a SQLite database in `~/.n8n`. Good for development and demos. That is what `demo/demo.sh` starts.
2. **Docker, self-hosted** (what a company runs). One container for n8n, one for Postgres, and for scale one for Redis. The database is Postgres instead of SQLite because SQLite cannot be shared by several processes. Same workflows, same JSON, just a different `DB_TYPE` and connection settings. The compose file for that is `docker-compose.yml` here, and `demo/docker-demo.sh` runs it: n8n and Postgres in containers, workflows imported and published with the n8n CLI inside the container, then the same webhook events fired from the terminal. On my Mac I run Docker through Colima (no Docker Desktop).
3. **n8n Cloud.** n8n hosts it for you. Fastest start, least control. Blocks `$env` in expressions, no shell access, data lives on their servers. For a bank or a GSE that is usually a non-starter, which is why self-hosted matters.

"What if multiple users use one n8n instance?"

Two different questions hide in there.

Multiple people editing. One instance has user accounts (owner, admin, member), personal and shared projects, and credential sharing. Two people can be in the editor at once; n8n shows who else has a workflow open and the last save wins. Workflow history keeps versions. On the paid tiers there are RBAC projects and SSO/SAML.

Multiple executions at once. A single n8n process runs executions in parallel up to a concurrency cap, but one process is one CPU-bound Node.js event loop. Past a few hundred executions a minute you switch to **queue mode**:

- one or more **main** processes serve the editor, webhooks and the schedule
- Redis holds the queue
- N **worker** processes pull executions off the queue and run them
- Postgres holds workflows, credentials and execution logs, shared by all

A webhook request comes in on a main, gets pushed to Redis, a worker runs the workflow, and the result is written to Postgres. Workers are stateless; you add more when the queue grows. Everything in this repo runs unchanged in queue mode because none of it assumes a particular process handles it.

"Do you know what sticky sessions are?"

A **sticky session** is a load balancer rule: once a client has been sent to server A, keep sending that same client to server A (by cookie or source IP) instead of spreading its requests across A, B and C.

Where it comes up in n8n: when you run **more than one main process** behind a load balancer, the editor keeps a live push connection open (WebSocket or server-sent events) to whichever main it first hit, and that main is the one that knows about the browser session. If the load balancer bounces the next request to a different main, the editor loses live updates or sees stale state. So multi-main n8n needs sticky sessions on the load balancer for the UI, and `N8N_MULTI_MAIN_SETUP_ENABLED=true` so the mains coordinate through Redis.

Webhooks do **not** need stickiness. Any main (or a dedicated webhook processor) can accept a webhook and push it to the queue.

"What is a credential in n8n?"

A stored secret (API key, token, username and password, OAuth token) that a node uses to authenticate to an outside service. Stored encrypted in n8n's database with `N8N_ENCRYPTION_KEY`. Referenced by name and id from the workflow, never copied into it. Shown in this repo: `demo/prepare-demo.js` writes four credentials, `n8n import:credentials` encrypts them on import, and every HTTP Request node here picks one from a dropdown ("Generic credential type" for APIs without a dedicated node). Longer version: `docs/how-this-works.md`.

"What is a webhook? What is a cron job?"

Webhook: a URL I own that another system POSTs to when something happens. Cron: a schedule, written as five fields (`0 10 * * 1-5` is 10:00 Monday to Friday). Workflows 01 to 03 are webhooks, 04 is cron. `docs/how-this-works.md` has the long version with the payloads.

"Do you know the vector / linear algebra part?"

The part that matters for a chatbot over documents:

- Text goes through an **embedding model** and comes out as a vector: a list of numbers, say 1,536 of them, that encodes the meaning. Similar meanings give vectors that point the same direction.
- To compare two vectors you take their **cosine similarity**: dot product divided by the product of the lengths. It ranges from -1 to 1. **Close to 1 means "these two texts mean nearly the same thing."** 0 means unrelated. Cosine distance is `1 - similarity`, which is what pgvector's `<=>` operator returns.
- A **vector database** (pgvector inside Postgres, Pinecone, Qdrant) stores the vectors with an index so "find the 5 nearest to this one" is fast over millions of rows.
- **RAG** (retrieval-augmented generation) is: embed the question, find the nearest stored chunks, hand those chunks to the LLM as context, let it answer.

In this repo, workflow 08 turns each workflow's description into a vector and stores it in Postgres with pgvector; workflow 07 embeds the question, runs `ORDER BY embedding <=> $1 LIMIT 1`, and returns the closest workflow with its similarity score; workflow 06 is the chat window on top. The demo uses a small hashed bag-of-words vector computed in a Code node so it needs no embedding API, and the query returns the similarity so you can see the number. Swap the Code node for n8n's OpenAI or Cohere Embeddings node for real semantic vectors; the SQL does not change.

The math itself, so it is not hand-waved: for vectors **a** and **b**, `cos(θ) = (a · b) / (‖a‖ ‖b‖)`, where `a · b = Σ aᵢbᵢ` and `‖a‖ = √(Σ aᵢ²)`. The Code node in workflow 07 computes exactly that in six lines before handing the vector to Postgres.

"Show me a chatbot that calls n8n and returns a workflow"

Workflow 06. The Chat Trigger node gives you a hosted chat page. Each message goes to an Execute Workflow node that calls workflow 07 as a sub-workflow (the n8n way to make one workflow call another and wait for its answer). 07 looks the question up in Postgres and returns the matching workflow. 06 formats the reply, inserts the question, answer and similarity score into the `chat_log` table, and sends the reply back to the chat window.

Open it in the demo: `http://localhost:5678/webhook/<id>/chat`, printed by `demo/demo.sh`, or click "Open chat" on the Chat Trigger node.

"Put it into Postgres"

Three ways this repo writes to Postgres, all in the Postgres node with parameterized SQL (`$1, $2`, never string concatenation, which is how you avoid SQL injection):

- `INSERT` a chat log row (workflow 06)
- `INSERT ... ON CONFLICT DO UPDATE` to upsert a workflow's vector (workflow 08)
- `SELECT ... ORDER BY embedding <=> $1::vector LIMIT 1` for nearest neighbour (07)

The table definitions are in `demo/schema.sql`.